

SP-3a Coverage Exemption Ledger

Per Constitutional clause 6.D (Behavioral Coverage Contract), every uncovered line in code authored under SP-3a is listed here with reason, reviewer, and date. Blanket waivers are forbidden.

See also: Seventh Law clause 5 (Recurring Bluff Hunt). Every entry in this ledger is eligible to be picked by the recurring 5-file random sample at phase wrap. An exemption that cannot survive the bluff hunt — i.e. the exempted code path is silently exercised by a test the hunt mutates — is a Seventh Law incident in waiting and **MUST** be audited rather than left as a paper-only exemption. The ledger is necessary, not sufficient: the recurring hunt is the live falsifiability check.

This file was seeded ahead of Task 0.7 by Task 0.2 (TestBookmarksRepository stub bluff) because that task needed an entry to land. Task 0.7 will append the Phase-0 summary section without re-creating the file.

Format

File	Lines	Reason	Reviewer	Date	PR
------	-------	--------	----------	------	----

Entries

File	Lines	Reason
core/testing/src/main/kotlin/lava/testing/repository/TestBookmarksRepository.kt	all (every method body is TODO("Not yet implemented")), except clear() which is a no-op)	Complete stub fake. Full behavior implementation against BookmarksRepository (Room BookmarkDao, @PrimaryKeys, JSONArray-encoded newTopics, first SP-3a consumer of BookmarksRepository, anticipated SP-3a Phase 4 (to be implemented) later. Stub status is locked in place. test TestBookmarksRepositoryStub stub is replaced with a real implementation. assertions will start failing as soon as we get to (a) remove this row, and (b) test with a true equivalence

File	Lines	Reason
core/testing/src/main/kotlin/lava/testing/service/TestAuthService.kt	all	TestEndpointsRepositoryEquivalentMenuViewModelTest constructs ClearBookmarksUseCase(TestBookmarks) but never triggers a code path for the method (no MenuAction.ClearBookmarks) the stub is latent rather than a Bluff Pact Third Law violation of falsifiability rehearsal record sp3a-bluff-audit/0.2-test-bookmarks Fake has zero test consumers as of 2026-04-30 (verified via grep --include='*.kt' core/ feature-declaration site — empty response on the Phase 0 audit, equivalent to the first consumer (anticipation) RuTracker decoupling tests AuthenticatableTracker feature the first consumer is added, written TestAuthServiceEquivalenceTest TestEndpointsRepositoryEquivalent AuthServiceImpl token persistence storage, login/logout state transition observation). Anti-Bluff Pact the fake cannot mislead any test consumer until a consumer materializes recorded in .lava-ci-evidence/test-auth-service.json. Equivalence already verified by contract tests in core/domain/src/test/kotlin/lava/testing/contract/TestInfrastructureContractTest // TestLocalNetworkDiscoveryService (emit contract section) covers the behaviour surface — non-susceptible to ordering, cross-scheduler resolution Third-Law bluff that the fake rendezvous-Channel() form works in core/domain/src/test/kotlin/lava/testing/contract/TestLocalNetworkDiscoveryContractTest (@Test methods) covers the rendezvous logic via matchesService SP-3.4 cross-match regression tests NOT implement suffix scoping post-resolution channel that the resolved DiscoveredEndpoint is the scoping happens at the NSD that the fake intentionally by LocalNetworkDiscoveryService "emit DiscoveredEndpoints", not
core/testing/src/main/kotlin/lava/testing/service/TestLocalNetworkDiscoveryService.kt	n/a — fake is a post-resolution injection point	

File	Lines	Reason
		type"). No new test required documents the audit's resolution in .lava-ci-evidence/sp3a-bluff-network-discovery.json.

Latent findings

The following are NOT coverage exemptions — they are known-risk observations surfaced during the Phase 0 audit. They do NOT block Phase 1 (Foundation), but each is a future-facing tripwire that MUST be revisited before its respective trigger condition materialises.

LF-1 — MenuViewModelTest holds TestBookmarksRepository without exercising it (linked: Task 0.2)

Location: feature/menu/src/test/kotlin/lava/menu/MenuViewModelTest.kt:115-118

Observation: the test constructs TestBookmarksRepository() and wires it into ClearBookmarksUseCase, which is then injected into MenuViewModel. As of 2026-04-30 no @Test method in this file dispatches a MenuAction whose handler invokes a stubbed method on the repository — i.e. the bookmarks fake is held but never touched. The instant a test is added that drives, for example, a MenuAction.ClearBookmarks flow, the stub will throw NotImplementedError at the first stubbed call site (getBookmarks(), saveBookmark(...), etc.) and the test will fail at runtime, not at compile time.

Mitigation trigger: when the first MenuViewModel test path that exercises bookmarks is added (anticipated during SP-3a Phase 4 tracker settings work or earlier feature work), the author MUST either:

1. Implement TestBookmarksRepository against the TestEndpointsRepositoryEquivalenceTest pattern (full behavioural equivalence to BookmarksRepositoryImpl), and remove the corresponding ledger entry above; or
2. Document why the new test path does not need a real fake (e.g. it uses a purpose-built local fake), and leave the ledger entry intact.

Audit linkage: Task 0.2 evidence at .lava-ci-evidence/sp3a-bluff-audit/0.2-test-bookmarks-repository.json.

LF-2 — TestHealthcheckContract is currently a future-facing tripwire (linked: Task 0.6)

Location: lava-api-go/tests/contract/
healthcheck_contract_test.go::TestHealthcheckContract

Observation: Constitutional clause 6.A (real-binary contract tests) was authored in response to the lava-api-go --http3 healthcheck regression of 2026-04-29, where docker-compose.yml invoked healthprobe --http3 https://localhost:8443/health against a binary that only registered -url/-insecure/-timeout. The contract test extracts the healthcheck command from docker-compose.yml, recovers the registered flag set from the binary's Usage output, and asserts strict subset. As of 2026-04-30 the lava-api-go service block in docker-compose.yml has no healthcheck: block at all (the bug was triaged by removing the broken healthcheck rather than fixing the flag). Consequently the contract test's "extract command" step finds an empty command and the subset check is trivially satisfied — the test passes vacuously.

The test is still load-bearing as a tripwire: the moment a healthcheck is re-introduced for lava-api-go, this contract test will validate it against the actual flag set of the healthprobe binary at the pinned hash. The falsifiability rehearsal recorded in Task 0.6 confirmed the test fails when a bogus --http2 flag is added to the compose healthcheck.

Mitigation trigger: before re-introducing any healthcheck: block on the lava-api-go service in docker-compose.yml, the author MUST:

1. Run `go test ./tests/contract/... -run TestHealthcheckContract` against the new compose file and confirm the test now exercises the subset check non-vacuously (i.e. the extracted command is non-empty).
2. Add a falsifiability rehearsal sub-test asserting that an unregistered flag in the new healthcheck command causes the contract checker to fail.
3. Record the rehearsal in a new evidence file under `.lava-ci-evidence/` with the same Test/Mutation/Observed/Reverted protocol used by Task 0.6.

Audit linkage: Task 0.6 evidence at `.lava-ci-evidence/sp3a-bluff-audit/0.6-healthprobe-contract.json`.

LF-3 — Tracker chain compiles to JVM 21 while Android targets JVM 17 (linked: Phase 2 Section A) — RESOLVED in Phase 2 Section E wrap-up

Resolution (2026-04-30): mitigation trigger #1 applied. Tracker-SDK submodule commit b779fda adds subprojects { ... jvmTarget = JVM_17 } to submodules/tracker_sdk/build.gradle.kts,

pinning every SDK subproject to JVM 17. Lava's three JVM 17→21 overrides reverted (KotlinTrackerModuleConventionPlugin, core/tracker/api/build.gradle.kts, core/tracker/testing/build.gradle.kts). Submodule pin updated in Lava to b779fda. Build verified GREEN end-to-end:

```
:core:tracker:api, :core:tracker:testing, :core:tracker:ru:tracker:test  
all compile and test against JVM 17 toolchain.
```

Submodule commit landed locally on branch sp3a-jvm17-target; mirror push to GitHub + GitLab is a separate operator action (per Decoupled Reusable Architecture rule "Submodule fetch/pull is an EXPLICIT operator action").

The original observation below is preserved for forensic context.

Locations:

- buildSrc/src/main/kotlin/KotlinTrackerModuleConventionPlugin.kt
- core/tracker/api/build.gradle.kts
- core/tracker/testing/build.gradle.kts

Observation: Phase 2 Section A (Task 2.5) required the new :core:tracker:ru:tracker module to compile via lava.kotlin.tracker.module. At HEAD the Tracker-SDK composite-build's :api/:mirror/:registry/:testing projects default to the JDK 21 toolchain and emit JVM 21 class files. The Lava-side tracker convention plugin previously targeted JVM 17, which produced "class file has wrong version 65.0, should be 61.0" failures the moment any Lava module on the JVM-17 chain (e.g. :core:tracker:api, :core:tracker:testing) tried to consume an SDK type. To unblock Section A's required BUILD SUCCESSFUL acceptance gate, the implementer bumped JVM target to 21 in three places: the new tracker convention plugin, and the two pure-Kotlin Lava-side tracker modules that were previously on JVM_17 via lava.kotlin.library. The rest of the codebase (Android modules via KotlinAndroid.kt, regular Kotlin libraries via KotlinLibraryConventionPlugin.kt) remains JVM_17 — the APK still targets JVM 17 bytecode.

Why this is a latent finding rather than a fix: The plan does not authorize a JVM-target bump and the cleaner resolution is upstream — set kotlin { jvmToolchain(17) } on each Tracker-SDK Gradle module. That requires a new Tracker-SDK pin and a fresh 4-upstream mirror push. Deferred so the rename can land cleanly in Phase 2 Section A.

Mitigation trigger: before Section F (Tasks 2.28-2.31, Hilt + LavaTrackerSdk wiring) lands the tracker chain into the Android :app Hilt module, the author MUST verify one of:

1. The Tracker-SDK pin has been updated upstream to target JVM 17 in its convention/module Gradle scripts, the new pin

recorded in submodules/tracker_sdk/, and the three JVM_21 overrides reverted to JVM_17 in KotlinTrackerModuleConventionPlugin.kt, core/tracker/api/build.gradle.kts, core/tracker/testing/build.gradle.kts.

2. The Android build (AGP/D8) is configured to accept JVM 21 input bytecode, and the rest of the codebase is bumped to JVM 21 for consistency. (Larger blast radius — only choose this path if upstream Tracker-SDK is sticky on JVM 21.)

If neither condition is met before Section F, :app:assembleDebug will fail at D8 dexing the moment the tracker registry lands on the dependency graph.

Audit linkage: Phase 2 Section A spec compliance review (commit landing the LF-3 entry).

LF-5 — RuTrackerDescriptor declares UPLOAD + USER_PROFILE without feature interfaces (linked: Phase 2 Section C) — RESOLVED 2026-04-30

Status: RESOLVED by commit lf-5-resolved on 2026-04-30. Resolution chose mitigation option 2 (drop the two capabilities) rather than option 1 (add the feature interfaces) because the legacy UploadTorrentUseCase and GetCurrentProfileUseCase plumbing is not consumed by any SP-3a feature ViewModel — adding the interfaces would be scope creep beyond the SP-3a foundation. Re-introducing either capability is appropriately scheduled for a future SP (SP-3a-bridge or later) where a feature ViewModel actually needs it.

Location: core/tracker/rutacker/src/main/kotlin/lava/tracker/rutacker/RuTrackerDescriptor.kt

Observation (historic): Phase 2 Section B (Task 2.6) populated RuTrackerDescriptor.capabilities with the full 12-capability set including TrackerCapability.UPLOAD and TrackerCapability.USER_PROFILE. As of Phase 2 Section C end-of-task, the new :core:tracker:api package did NOT define UploadableTracker or ProfileTracker feature interfaces. Consequently RuTrackerClient.getFeature<T>() could not return a non-null impl for those two capabilities — the KClass<T> for the missing interfaces did not exist as a dispatch key in the when switch.

Constitutional clause 6.E literal: "capability declared in descriptor ⇒ this returns non-null". The original code SATISFIED this letter-of-the-law because no caller could construct getFeature(UploadableTracker::class) — the type didn't exist. But the descriptor was making a forward-looking claim that no impl backed. Treated as a latent finding rather than an immediate violation.

Resolution (2026-04-30):

- RuTrackerDescriptor.capabilities reduced from 12 to 10 entries.
- UPLOAD and USER_PROFILE removed.
- RuTrackerDescriptorTest assertion updated from 12 capabilities to 10 capabilities and a new test LF-5 RESOLVED – UPLOAD and USER_PROFILE are NOT declared (no feature interface) added.
- core/tracker/rutracker/README.md capability matrix updated.
- The dormant scrapers (RuTrackerInnerApi.uploadTorrent, getProfile) and their wrapping UseCases continue to ship as legacy plumbing in this module but are no longer advertised through the SDK.

Audit linkage: Phase 2 Section C spec + code-quality review (commit landing the LF-5 entry); Phase 5 wrap follow-up (commit lf-5-resolved).

LF-6 — TorrentItem.sizeBytes permanently null for rutracker (linked: Phase 2 Section D) — RESOLVED 2026-04-30

Status: RESOLVED by commit lf-6-resolved on 2026-04-30. Resolution chose mitigation option 2 (parse the formatted display string in the forward mapper). A new internal helper RuTrackerSizeParser lives next to the forward mappers in core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/mapper/ and handles GB/MB/KB/B/TB suffixes with optional decimal (period or comma) at binary multipliers ($1\text{ GB} = 2^{30}$) — matching what the legacy rutracker scraper's formatSize helper renders, so a render-then-parse round-trip is bitwise correct (modulo Long truncation of the truncated Double value, which is the same precision the formatted display string carries).

Locations:

- core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/mapper/SearchPageMapper.kt
- core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/mapper/CategoryPageMapper.kt
- core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/mapper/TopicMapper.kt

Observation (historic): The new tracker-api model TorrentItem exposes sizeBytes: Long?. The legacy rutracker scraper delivers torrent size as a pre-formatted display string (e.g. "4.7 GB"), and the raw byte count is discarded before the DTO reaches the forward mapper. Consequently every TorrentItem produced by the rutracker forward mappers HAD sizeBytes = null. The display string was preserved in metadata["rutracker.size_text"] so UI could render it, but any consumer of Searchable/Browsable/Topic

features that relied on `sizeBytes` for filtering, sorting, or comparison against a threshold silently received `null` for every `rutracker` row.

Resolution (2026-04-30):

- New `RuTrackerSizeParser` (internal object) in `core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/mapper/RuTrackerSizeParser.kt`. Tolerates null/blank input by returning null; tolerates U+00A0 between number and unit; supports B/KB/MB/GB/TB (case-insensitive); accepts comma-decimal as well as period-decimal.
- `SearchPageMapper.toTorrentItem` (the `TorrentDto` extension) now calls the parser; `TopicMapper.toTorrentItem` (the `TopicPageDto` extension) does the same. `CategoryPageMapper`'s `TopicDto` branch is intentionally unchanged — `TopicDto` has no `size` field upstream.
- Tests: `RuTrackerSizeParserTest` (15 cases covering integer GB, decimal GB period+comma, MB, KB, B, integer TB, decimal TB Double-precision truncation, case-insensitive, null, blank, garbage, no-whitespace, non-breaking-space). `SearchPageMapperTest` and `TopicMapperTest` now positively assert non-null `sizeBytes` on the fixtures with a `size` field.

Audit linkage: Phase 2 Section D combined review (commit landing the LF-6 entry); Phase 5 wrap follow-up (commit `lf-6-resolved`).